DDJ-ERGO -> Mixxx Mapping

Technical Architecture, Modal Logic, LED Feedback, and Operational Constraints

A contributor-oriented design paper for collaborative development and future extension of the DDJ-ERGO controller mapping for Mixxx.

Document type	Technical design paper
Scope	MIDI architecture, XML/JS split, state model, pad modes, loop engine, LED semantics, routing, browser logic
Target use	Repository documentation and shared reference for maintainers and contributors
Prepared	April 2026

Copyright (c) 2026 DDJ-ERGO Mixxx Mapping Contributors. This documentation may be shared and adapted under the Creative Commons Attribution 4.0 International (CC BY 4.0) license, unless the repository specifies otherwise. Pioneer DJ, DDJ-ERGO, and Mixxx remain the property or trademarks of their respective owners.

Contents

1. Purpose . . . 3
2. MIDI Architecture . . . 3
    2.1 Channel and status layout . . . 3
    2.2 14-bit continuous controls . . . 3
3. XML vs JavaScript Responsibilities . . . 4
4. Internal State Model . . . 4
5. Pad Mode System . . . 5
    5.1 Mode selectors . . . 5
    5.2 Pad note map . . . 5
    5.3 Hotcue mode . . . 5
    5.4 Loop mode . . . 6
    5.5 Looproll mode . . . 6
6. Modal Constraints and Guard Rails . . . 7
7. Loop Engine Logic . . . 7
    7.1 Discrete loop sizes . . . 7
    7.2 Dual behavior of the loop encoder . . . 7
    7.3 Asynchronous loop-size refresh and loop-pad truth model . . . 8
8. Jump, Key, and Rate Logic . . . 8
    8.1 Beatjump sizes . . . 8
    8.2 Key encoder and rate range . . . 8
9. LED Feedback System . . . 9
10. FX and Routing Layer . . . 9
    10.1 FX button layout . . . 9
    10.2 FX channel routing and mix mode . . . 10
11. Browser, Preview, and Track Loading . . . 10
    11.1 Context-aware browser encoder . . . 10
    11.2 Preview and load workflow . . . 10
12. Recording Logic . . . 10
13. Engineering Summary . . . 11
14. Developer Notes . . . 11
15. Planned Reserved Mode: Stems Control for Mixxx 2.6 . . . 11

The document is intentionally written as a contributor-facing technical reference: compact enough to remain maintainable, but detailed enough to preserve behavioral intent across future revisions.

1. Purpose

This mapping is not a simple one-to-one MIDI assignment. It is a layered control system combining XML MIDI bindings, JavaScript behavioral logic, per-deck internal state, custom LED semantics, and performance-oriented guard rails. The objective is to keep controller interaction technically predictable and musically coherent, especially in the domains of cue triggering, loop control, roll gestures, browser context switching, FX routing, and safe deck loading.

The document is intended for collaborative repository work: it records not only what the mapping does, but why its current constraints exist, so that future contributors can extend the controller without breaking established performance logic.

2. MIDI Architecture

2.1 Channel and status layout

Table 1 - Global MIDI channel structure

Status	Type	Scope	Description
0x90-0x93	Note On	Deck 1-4	Deck buttons
0xB0-0xB3	CC	Deck 1-4	Deck encoders, pitch, continuous controls
0x94-0x95	Note On	FX Unit 1-2	FX buttons
0xB4-0xB5	CC	FX Unit 1-2	FX knobs
0x96	Note On	Global	PFL, load, eject, preview, record, FX routing
0xB6	CC	Global mixer	Gain, EQ, volume, crossfader, browser, headphones

The controller is deck-regular: identical note and CC numbers are reused across all four decks, while the status byte determines which deck receives the action. The script reconstructs deck identity either from the Mixxx group string or from the low nibble of the MIDI status byte. This keeps the JavaScript handlers generic and limits code duplication.

2.2 14-bit continuous controls

Several analog controls are mapped in 14-bit mode, using standard MSB/LSB pairing. This is particularly relevant for DJ performance because it improves pitch precision, EQ smoothness, gain resolution, and the tactile feel of crossfader and line-fader response.

Table 2 - Representative 14-bit controls

Control	Channel	MSB	LSB
Pitch deck 1	0xB0	0x00	0x20
Pitch deck 2	0xB1	0x00	0x20
Master gain	0xB6	0x00	0x20
Head mix	0xB6	0x01	0x21
Head gain	0xB6	0x02	0x22
EQ High Ch1	0xB6	0x07	0x27
EQ Mid Ch1	0xB6	0x08	0x28
EQ Low Ch1	0xB6	0x09	0x29
Volume Ch1	0xB6	0x13	0x33
Crossfader	0xB6	0x1F	0x3F

3. XML vs JavaScript Responsibilities

The mapping is deliberately split into two layers. The XML layer handles direct MIDI-to-ControlObject routing, declaration of deck and global controls, straightforward LED bindings, and delegation of advanced cases to JavaScript. The JavaScript layer implements modal behavior, loop and roll logic, internal state tracking, LED blink routines, state gating, browser context switching, and post-load recovery logic.

In short: XML defines routing; JavaScript defines semantics. This split is essential for maintainability because it keeps simple controls declarative while isolating stateful behavior in code.

4. Internal State Model

Table 3 - Core internal state

Variable	Scope	Purpose
_loopSizes	global	Discrete loop size table
_jumpSizes	global	Discrete beatjump size table
_loopIdx[4]	per-deck	Current logical loop size index
_jumpIdx[4]	per-deck	Current logical jump size index
_padMode[4]	per-deck	Current pad mode
_blinkTimer[4]	per-deck	Active loop-pad blink timer
_blinkState[4]	per-deck	Current blink phase
_blinkPadNote[4]	per-deck	Pad note representing the active loop
_recBlinkTimer	global	Recording blink timer
_recBlinkState	global	Recording LED phase

This internal model does not replace Mixxx runtime state. Instead, it acts as a controller-side interaction layer. When required - especially in loop representation - the script reconciles its own expectations against the actual

Mixxx ControlObject values to avoid false LED feedback.

5. Pad Mode System

5.1 Mode selectors

Table 4 - Pad mode buttons

Note	Mode	Status
0x3C	hotcue	active
0x3E	loop	active
0x40	looproll	active
0x42	reserved	intentionally unused in current version

Only one pad mode is active per deck at a time. The active mode button LED is lit while the others are off. The reserved slot is kept in the modal architecture on purpose and is not treated as a dead mapping address.

5.2 Pad note map

Table 5 - Main pads and shifted pads

Function	Note
Pad 1	0x2E
Pad 2	0x2F
Pad 3	0x30
Pad 4	0x31
Shift Pad 1	0x5F
Shift Pad 2	0x60
Shift Pad 3	0x61
Shift Pad 4	0x62

The shifted pads are hardware-distinct notes rather than a software overlay. The design therefore combines two mechanisms: software modal reinterpretation for the pad bank, and hardware-level alternate notes for secondary actions.

5.3 Hotcue mode

Table 6 - Hotcue pad mapping

Pad	Action
Pad 1	hotcue_1_activate
Pad 2	hotcue_2_activate
Pad 3	hotcue_3_activate
Pad 4	hotcue_4_activate

Table 7 - Shifted hotcue clear mapping

Shift pad	Action
Shift 1	hotcue_1_clear
Shift 2	hotcue_2_clear
Shift 3	hotcue_3_clear
Shift 4	hotcue_4_clear

In Hotcue mode, pad LEDs reflect hotcue slot occupancy rather than momentary pad presses. From a DJing perspective, this is the correct semantic choice: the hardware visualizes track structure and cue availability, not transient finger contact.

5.4 Loop mode

Table 8 - Loop pad sizes

Pad	Loop size index	Loop size
Pad 1	3	0.25
Pad 2	4	0.5
Pad 3	5	1
Pad 4	6	2

In Loop mode, pads behave as size selectors, loop activators, and toggles. Pressing the pad that already represents the active loop closes that loop using reloop_exit; pressing a different pad closes the current loop, writes the new beatloop_size, activates the new loop, and reassigns the blinking state to the new pad.

5.5 Looproll mode

Table 9 - Looproll pad sizes

Pad	Roll size
Pad 1	0.25
Pad 2	0.5
Pad 3	1
Pad 4	2

Looproll is strictly press/release based. Pressing a pad activates the roll, releasing it deactivates the roll. The mapping therefore treats looproll as a temporary performance gesture rather than a latched loop state.

6. Modal Constraints and Guard Rails

Table 10 - Mode and action constraints

Constraint	Condition	Result
Cannot enter looproll	loop_enabled = 1	mode change denied
Cannot exit looproll	mode = looproll and loop_enabled = 1	mode change denied
Loop knob turn blocked	looproll active and roll engaged	input ignored
Loop knob press blocked for loop creation	mode = looproll	input ignored
loop_in / loop_out blocked	mode = looproll	input ignored
Key knob blocked	keylock = 0	input ignored
Key reset blocked	keylock = 0	input ignored
Track load blocked	deck is playing	input ignored
Reserved mode blocked	current implementation	ignored

These constraints are intentional. The mapping favors denial over ambiguity: invalid or unsafe actions are blocked rather than translated into unpredictable side effects. When a mode change is rejected, the requested mode LED flashes rapidly, clearly indicating that the action was understood but refused.

7. Loop Engine Logic

7.1 Discrete loop sizes

Table 11 - _loopSizes

Index	Value	Index	Value	Index	Value
0	0.03125	5	1	10	32
1	0.0625	6	2	11	64
2	0.125	7	4	12	128
3	0.25	8	8	13	256
4	0.5	9	16	14	512

The default per-deck loop index is 7, corresponding to a 4-beat loop. The pads expose only a performance-oriented subset of this table; the encoder accesses the broader range.

7.2 Dual behavior of the loop encoder

The loop encoder has two operating paths. If no active manual loop is detected, it navigates the discrete loop table by updating _loopIdx and writing the resulting beatloop_size. If a manual loop is active - meaning the real beatloop_size does not match the internally expected size - the script treats that loop as manually defined and scales it using loop_double or loop_halve instead of immediately forcing a discrete table value.

This distinction preserves the musical intent of manually placed in/out loops. It prevents the encoder from destroying an existing manual loop structure just because the controller-side index is not aligned with Mixxx runtime state.

7.3 Asynchronous loop-size refresh and loop-pad truth model

After loop_double or loop_halve, the script delays LED refresh by 50 ms so that Mixxx has time to propagate the updated beatloop_size. In Loop mode, the blinking pad is determined from the actual beatloop_size rather than only from _loopIdx. This is critical: if the active loop was created manually and does not belong to the four pad sizes, the mapping intentionally shows no active loop pad rather than showing the wrong one.

8. Jump, Key, and Rate Logic

8.1 Beatjump sizes

Table 12 - _jumpSizes

Index	Value
0	0.5
1	1
2	2
3	4
4	8
5	16
6	32

The default beatjump size is index 3, corresponding to 4 beats. The jump encoder selects beatjump_size, while dedicated backward and forward buttons execute the movement and emit a short event flash on their respective LEDs.

8.2 Key encoder and rate range

The key encoder is gated by keylock. Clockwise rotation triggers pitch_up, counterclockwise rotation triggers pitch_down, and pressing the encoder resets the key to the original track key. If keylock is disabled, the control is intentionally silent. This prevents tonal commands from operating outside a pitch-preserving workflow.

Table 13 - Rate range sequence

Step	Range
1	0.10
2	0.16
3	0.24
4	0.50
5	1.00

Pitch range cycles through these values with wrap-around logic. Confirmation is shown by flashing LED 0x1B, deliberately leaving the keylock LED untouched so that its on/off semantics remain clean.

9. LED Feedback System

Table 14 - LED feedback categories

Category	Meaning	Example
Static state	Persistent software state	keylock, sync, hotcue existence
Dynamic state	Persistent but animated state	active loop pad blink, recording blink
Event feedback	Short transient confirmation	beatjump flash
Rejection feedback	Denied action	mode button rapid flash
Change acknowledgment	Accepted parameter or mode cycle	FX mix mode flash, rate range flash

Representative LED addresses

Note	Meaning
0x0B	Play
0x0C	Cue
0x10	Loop In
0x11	Loop Out
0x1A	Keylock
0x3C / 0x3E / 0x40 / 0x42	Pad mode LEDs
0x4C / 0x4D	Beatjump back / forward feedback
0x58	Sync enabled
0x5C	Sync master
0x66	Recording

The LED layer is semantic rather than uniform. Different blink patterns encode different classes of information: 250 ms loop-pad blink for active loop state, 500 ms blink for recording, 100 ms flash for beatjump events, rapid flash for denied mode changes, and short cycle flashes for accepted mode or parameter changes.

10. FX and Routing Layer

10.1 FX button layout

Table 15 - FX effect buttons

Note	Function
0x47	Effect 1
0x48	Effect 2
0x49	Effect 3

These notes are reused on two status channels: 0x94 for FX Unit 1 and 0x95 for FX Unit 2.

10.2 FX channel routing and mix mode

Table 16 - FX assignment buttons

Note	Routing
0x4C	FX Unit 1 -> Channel 1
0x4D	FX Unit 1 -> Channel 2
0x4E	FX Unit 1 -> Channel 3
0x4F	FX Unit 1 -> Channel 4
0x50	FX Unit 2 -> Channel 1
0x51	FX Unit 2 -> Channel 2
0x52	FX Unit 2 -> Channel 3
0x53	FX Unit 2 -> Channel 4

FX routing buttons are handled fully in JavaScript through an explicit lookup table. Each press toggles the corresponding group_[ChannelN]_enable state and updates the LED manually. The mix_mode button cycles between the two Mixxx FX mix modes and acknowledges the change with a short flash sequence before returning to its steady lit state.

11. Browser, Preview, and Track Loading

11.1 Context-aware browser encoder

The main browser encoder is dual-purpose. If PreviewDeck1 is stopped, it navigates the library using MoveUp and MoveDown. If PreviewDeck1 is playing, the same encoder becomes a preview scrubbing control by modifying playposition. A secondary encoder layer handles focus navigation into and out of browser hierarchy using MoveFocusForward and MoveFocusBackward.

11.2 Preview and load workflow

The preview button acts as a toggle: if preview is already active, it stops and ejects the preview deck; otherwise, it loads the selected track into PreviewDeck1, starts playback, and enables PFL. The deck load command is guarded so that no track can be loaded onto a playing deck.

When a new track is loaded, the script resets the controller-side performance state for that deck: loop index returns to its default, jump index returns to its default, pad mode returns to Hotcue, and any active loop-pad blink is cleared. After a 200 ms delay, the script reasserts beatloop_size, beatjump_size, and clears any phantom loop boundary state. This delay is necessary because Mixxx overwrites those values internally during track load.

12. Recording Logic

Table 17 - Recording behavior

State	LED behavior
Recording off	LED off
Recording on	Blinking at 500 ms

Recording uses a dedicated global LED on 0x96 / 0x66. The script manages recording feedback through its own independent timer so that recording blink logic remains separate from all per-deck blink logic.

13. Engineering Summary

From an implementation standpoint, the mapping is built around five principles: consistent deck abstraction, stateful JavaScript layered over declarative XML, strict modal separation between Hotcue, Loop, and Looproll workflows, semantic LED design, and protective guard rails that prioritize live-performance reliability over permissive but ambiguous behavior.

From a DJ-performance standpoint, the strongest aspects of the design are the explicit handling of loop versus looproll incompatibility, the distinction between discrete and manual loop manipulation, the visibility of cue-slot state, the reuse of the browser encoder as a preview transport control, and the safe deck-loading sequence that resets controller-side performance state without racing Mixxx internal load behavior.

14. Developer Notes

Any future extension should preserve the current design philosophy: do not overload LEDs with conflicting meanings, keep controller-side state distinct from Mixxx engine truth, prefer explicit state transitions over implicit side effects, and preserve rejection logic for invalid actions. New features should be introduced as structured subsystems rather than isolated button hacks.

15. Planned Reserved Mode: Stems Control for Mixxx 2.6

The currently unused reserved pad mode is intentionally preserved for a future stems-oriented workflow targeting the planned Mixxx 2.6 branch of the mapping. Its purpose is to provide direct hardware access to the stems panel, allowing the user to address and selectively enable or suppress individual song components such as vocals, drums, bass, and melodic or harmonic content, depending on the final Mixxx stems API and ControlObject model.

This would be a coherent extension of the current modal design. In Hotcue mode, pads address timeline positions; in Loop and Looproll modes, pads address time-domain repetition; in Reserved mode, the same pad bank would shift into source-domain control, turning the pads into selectors for track-layer activation or suppression rather than temporal performance triggers. Architecturally, the existing mode system already provides the required isolation, LED addressing, and per-deck state handling needed to support that future stems layer cleanly.